Keep Multiplying Found Values by Two

Company: Goldman Sachs
Year: Not specified
Difficulty: Easy
Topic: Arrays, HashSet
Source: LeetCode 2154 – Keep Multiplying Found Values by Two

LeetCode 2154 – Keep Multiplying Found Values by Two

Problem Description

You are given an integer array nums and an integer original.

You need to repeatedly search for original in nums.

The process is:

1. If original exists in nums, multiply it by 2.
2. Search for the new value again.
3. Continue as long as the current value is found.
4. If the current value is not found, stop and return it.

For example:

nums = [5,3,6,1,12]

original = 3

The process is:

3 → 6 → 12 → 24

24 is not present in the array, so the final answer is:

24

Input Format

- The first line contains two integers N and original — the number of elements in the array and the initial value to search for.
- The second line contains N space-separated integers denoting the array nums.

Output Format

- Print a single integer — the final value of original after repeatedly multiplying it by 2 whenever it is found in nums.

Constraints

- 1 ≤ N ≤ 1000
- 1 ≤ nums[i], original ≤ 1000

Example 1

Input:

N = 5

original = 3

nums = [5,3,6,1,12]

Output:

24

Explanation

3 is present:

3 × 2 = 6

6 is present:

6 × 2 = 12

12 is present:

12 × 2 = 24

24 is not present, so return:

24

Example 2

Input:

N = 3

original = 4

nums = [2,7,9]

Output:

4

Explanation

4 is not present in nums, so the process stops immediately.

Therefore:

4

is returned.

Approach to Solve This Problem

The simplest approach is to use a **HashSet**.

We need to repeatedly check whether the current value exists in nums.

Instead of searching through the entire array every time, we can put all elements into a HashSet.

A HashSet allows us to check whether a value exists in approximately O(1) time.

For example:

nums = [5,3,6,1,12]

original = 3

Store the elements:

Set = {5, 3, 6, 1, 12}

Now:

3 → found → 6

6 → found → 12

12 → found → 24

24 → not found

Therefore, return 24.

Java Solution

```
import java.util.*;

public class Main {

  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int original = sc.nextInt();

    HashSet<Integer> set = new HashSet<>();

    for (int i = 0; i < n; i++) {
      set.add(sc.nextInt());
    }
```

```
        while (set.contains(original)) {
            original = original * 2;
        }

        System.out.println(original);
    }
}
```

Solution:

1. **Read the input:** main() reads N and original, then reads N integers into the array and stores them in a HashSet, matching the Input Format.
2. **Create a HashSet:** Store all elements of nums in a HashSet so that we can quickly check whether the current value of original exists in the array.
3. **Search for the current value:** Use set.contains(original) to check whether the current value is present in nums.
4. **Multiply when found:** If original exists in the set, multiply it by 2 using original = original * 2.
5. **Repeat the process:** Continue checking the new value until original is no longer present in the HashSet.
6. **Stop when not found:** Once the current value is not present in nums, stop the loop. At this point, original contains the final required value.
7. **Print the result:** Print original as the final answer, matching the Output Format.

Complexity Analysis

- **Time Complexity:** O(N + K), where N is the number of elements in nums and K is the number of times original is found and doubled.
- **Space Complexity:** O(N) for storing the elements in the HashSet.